SIOC 221A _ Homework 2

Jingyi He (jih185@ucsd.edu)

Collaborators: none

AI use: I used Claude to draft the idea; I ran all the code, checked the results, and wrote the interpretations myself.

In [1]:
```
# Imports and setup
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import matplotlib.dates as mdates
from scipy import stats
%matplotlib inline
plt.rcParams['figure.figsize'] = (9, 4.5)   # default figure size for all plots

# My loading function (load_pier_data.py)
from load_pier_data import load_pier_data
```

Load the data

I compare **2015 and 2020**, read from the SCCOOS THREDDS server.

In [2]:
```
Y1, Y2 = 2015, 2020
data = {y: load_pier_data(y) for y in (Y1, Y2)}
VARS = {'temperature': ('temperature', '$^o$C'), 'pressure': ('pressure', 'dbar')}

# Gaps longer than one day, per year
for y, d in data.items():
    dt = np.diff(d['time']) / np.timedelta64(1, 'h')
    big = np.where(dt > 24)[0]
    print(f'{y}: {len(big)} gaps > 1 day', [f"{d['time'][i]} ({dt[i]:.0f} h)" for i in big])
```

```
    for v in VARS:
        print(f'   {v}: {np.isnan(d[v]).sum()} NaN values')
```

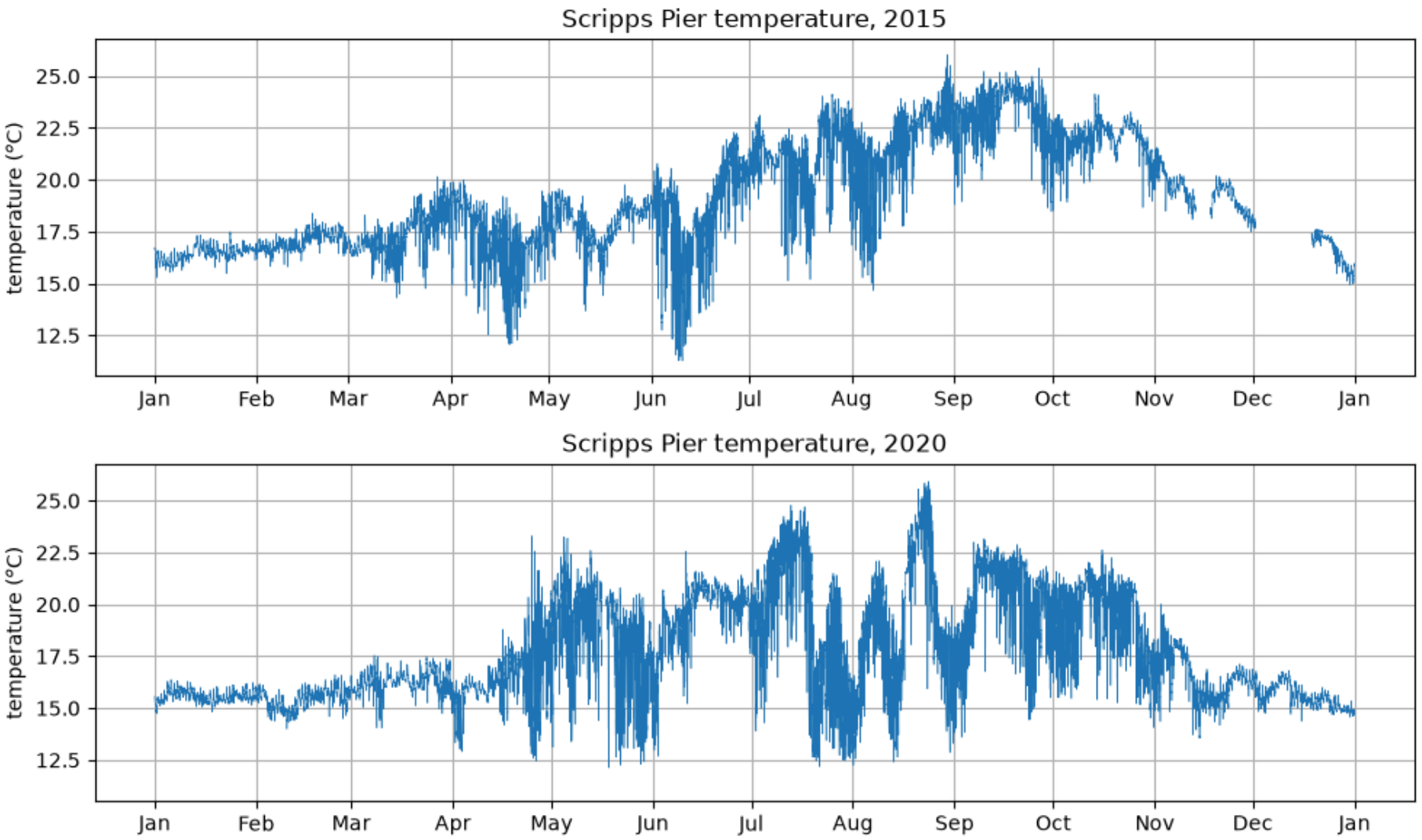
```
2015:   82237 samples, 2015-01-01T00:00:05 to 2015-12-31T23:56:18, median dt = 6.0 min
2020:  130442 samples, 2020-01-01T00:00:32 to 2020-12-31T23:57:35, median dt = 4.0 min
2015: 2 gaps > 1 day ['2015-11-13T17:57:56 (108 h)', '2015-12-01T20:29:11 (410 h)']
   temperature: 0 NaN values
   pressure: 0 NaN values
2020: 1 gaps > 1 day ['2020-05-16T10:56:54 (33 h)']
   temperature: 0 NaN values
   pressure: 0 NaN values
```

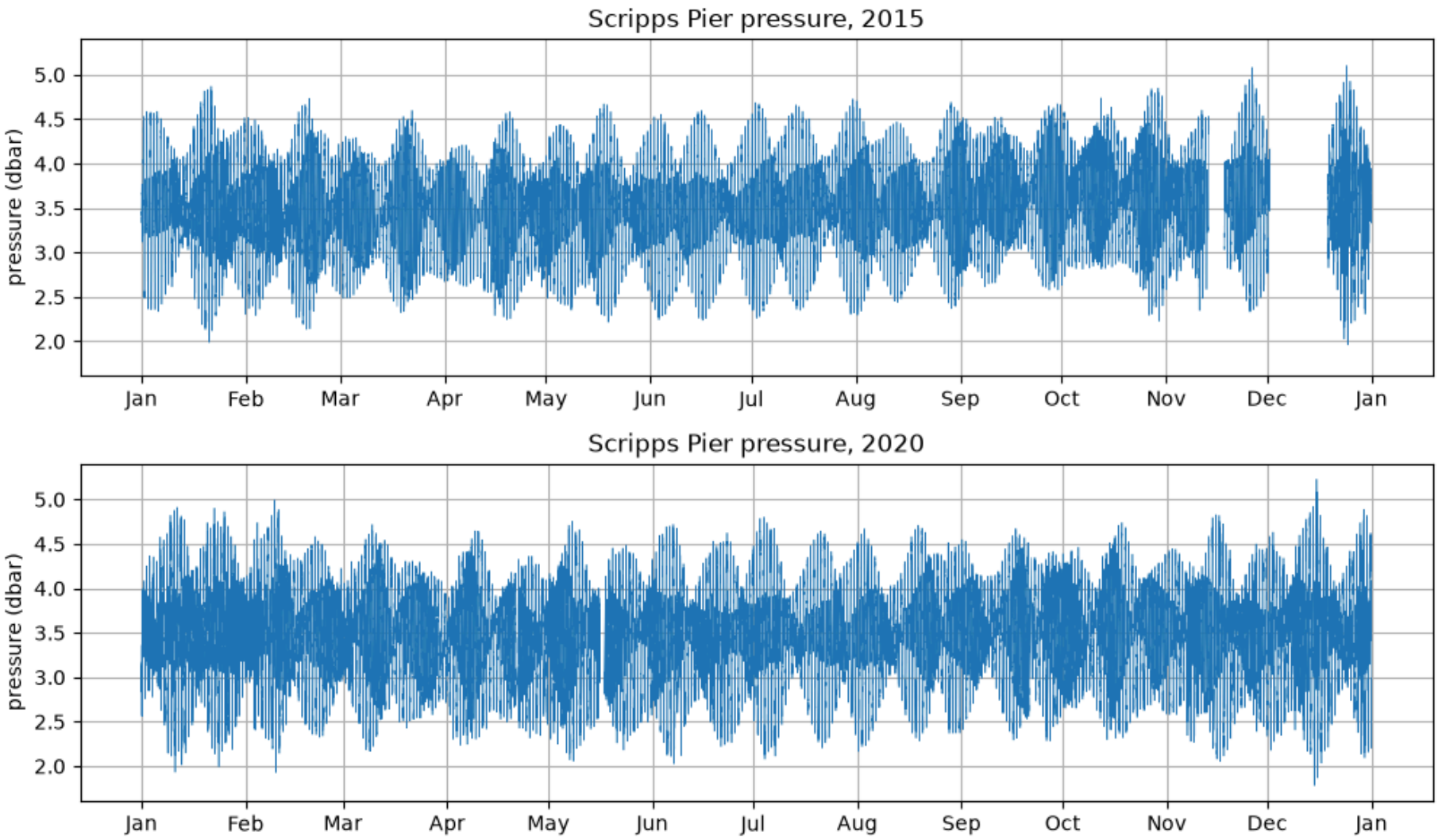
1. Visual evaluation

In [3]:

```
def insert_gaps(t, y, max_gap=np.timedelta64(1, 'D')):
    """Insert NaN at gaps longer than max_gap so the plot shows a break."""
    i = np.where(np.diff(t) > max_gap)[0] + 1
    return np.insert(t, i, t[i - 1] + np.timedelta64(1, 's')), np.insert(y, i, np.nan)

for v, (label, unit) in VARS.items():
    fig, axes = plt.subplots(2, 1, figsize=(10, 6), sharey=True)
    for ax, y in zip(axes, (Y1, Y2)):
        t, x = insert_gaps(data[y]['time'], data[y][v])
        ax.plot(t, x, linewidth=0.4)
        ax.xaxis.set_major_locator(mdates.MonthLocator())
        ax.xaxis.set_major_formatter(mdates.DateFormatter('%b'))
        ax.set_ylabel(f'{label} ({unit})')
        ax.set_title(f'Scripps Pier {label}, {y}')
        ax.grid(True)
    plt.tight_layout()
    plt.show()
```


Scripps Pier temperature, 2015
temperature (°C)
25.0
22.5
20.0
17.5
15.0
12.5
Jan
Feb
Mar
Apr
May
Jun
Jul
Aug
Sep
Oct
Nov
Dec
Jan
Scripps Pier temperature, 2020
temperature (°C)
25.0
22.5
20.0
17.5
15.0
12.5
Jan
Feb
Mar
Apr
May
Jun
Jul
Aug
Sep
Oct
Nov
Dec
Jan



1. Observations

Coverage. Both records cover the full year. 2015 is sampled every 6 min and 2020 every 4 min. 2015 has a 17-day gap in early December and a short gap in mid-November. 2020 is nearly continuous.

Temperature. Both years show a seasonal cycle, but it is not a simple sinusoid. Winter is calm in both years. 2015 is about 1–1.5 °C warmer in winter. From April to October, temperature swings by 5–10 °C over hours to days, with cold events down to 12 °C. These are likely caused by upwelling and internal waves. Summer variability is larger in 2020. The biggest difference is in the fall: 2015 stays at 22–25 °C through October, while 2020 cools to 15–17 °C. This matches the 2015 marine heatwave and El Niño. Variance is much larger in summer than in winter, so temperature is not stationary.

Pressure. Pressure is dominated by tides in both years. The mean is about 3.5 dbar and the range is 2–5 dbar. A 14-day spring–neap cycle is clear. Tidal ranges are largest near the solstices. The two years have the same mean level, so the sensor depth did not change much. There is a brief discontinuity in mid-May 2020.

2. Means

Standard error of the mean: $\sigma/\sqrt{N}$. I compare full-resolution data with data subsampled once per day. The two means are consistent if their difference is within about 2 combined standard errors, $z = (\bar{x}_1 - \bar{x}_2)/\sqrt{SE_1^2 + SE_2^2}$.

In [4]:
```
def daily_subsample(d, v):
    """One sample per day: the first valid sample of each day."""
    s = pd.Series(d[v], index=pd.to_datetime(d['time'])).dropna()
    return s.resample('D').first().dropna().values

def get_series(y, v, mode):
    x = data[y][v]
    return x[~np.isnan(x)] if mode == 'full' else daily_subsample(data[y], v)

def mean_se(x):
    return x.mean(), x.std(ddof=1) / np.sqrt(len(x)), len(x)

rows = []
for v in VARS:
    for mode in ('full', 'daily'):
        m1, se1, n1 = mean_se(get_series(Y1, v, mode))
        m2, se2, n2 = mean_se(get_series(Y2, v, mode))
        z = (m1 - m2) / np.sqrt(se1**2 + se2**2)
        rows.append([v, mode, n1, m1, se1, n2, m2, se2, z, abs(z) < 2])
cols = ['variable', 'sampling', f'N {Y1}', f'mean {Y1}', f'SE {Y1}',
```

```
        f'N {Y2}', f'mean {Y2}', f'SE {Y2}', 'z', 'consistent (|z|<2)']
pd.DataFrame(rows, columns=cols).round(4)
```

Out[4]:

	variable	sampling	N 2015	mean 2015	SE 2015	N 2020	mean 2020	SE 2020	z	consistent (|z|<2)
0	temperature	full	82237	19.2543	0.0091	130442	17.7528	0.0071	130.3322	False
1	temperature	daily	345	19.5742	0.1439	366	18.3642	0.1369	6.0908	False
2	pressure	full	82237	3.5010	0.0017	130442	3.4649	0.0014	16.3187	False
3	pressure	daily	345	3.3559	0.0260	366	3.3044	0.0260	1.3976	True

2. Discussion

Full resolution. The means of both variables appear inconsistent, with |z| = 130 and 16. But the standard errors are unrealistically small. The formula σ/√N assumes independent samples. Consecutive 4–6 min samples are almost identical, so the effective number of degrees of freedom is much smaller than N.

Daily subsampling. Using one sample per day makes the standard errors 15–20 times larger. Pressure becomes consistent, with |z| = 1.4. Tides decorrelate within a day, so daily pressure samples are nearly independent. Temperature is still inconsistent, with |z| = 6.1. Temperature anomalies last for days to weeks, so daily samples are still correlated and the true error is larger. The 1.2–1.5 °C difference is likely real and matches the 2015 heatwave. The December gap in 2015 also biases its mean slightly warm.

Sampling bias. Daily subsampling also changes the means. Temperature rises by 0.3–0.6 °C and pressure drops by 0.15 dbar. I took the first sample of each day, at 00:00 UTC, which is late afternoon locally. Sampling at the same time every day aliases the daily cycle. It always catches warm afternoon temperatures and the same phase of the S2 and diurnal tides. Daily averages would avoid this bias.

3. Variance

Confidence interval for the variance from the χ^2 distribution: $\frac{(N-1)s^2}{\chi^2_{1-\alpha/2,N-1}} \leq \sigma^2 \leq \frac{(N-1)s^2}{\chi^2_{\alpha/2,N-1}}$ (95%). I also do an F-test on the ratio s_1^2/s_2^2.

In [5]:
```
def var_ci(x, alpha=0.05):
    n, s2 = len(x), x.var(ddof=1)
    lo = (n - 1) * s2 / stats.chi2.ppf(1 - alpha/2, n - 1)
    hi = (n - 1) * s2 / stats.chi2.ppf(alpha/2, n - 1)
    return s2, lo, hi, n

rows = []
for v in VARS:
    for mode in ('full', 'daily'):
        x1, x2 = get_series(Y1, v, mode), get_series(Y2, v, mode)
        s1, lo1, hi1, n1 = var_ci(x1)
        s2, lo2, hi2, n2 = var_ci(x2)
        F = s1 / s2
        # two-sided p-value of the F test
        p = 2 * min(stats.f.cdf(F, n1 - 1, n2 - 1), stats.f.sf(F, n1 - 1, n2 - 1))
        overlap = not (hi1 < lo2 or hi2 < lo1)
        rows.append([v, mode, s1, lo1, hi1, s2, lo2, hi2, F, p, overlap])
cols = ['variable', 'sampling', f'var {Y1}', 'lo', 'hi', f'var {Y2}', 'lo', 'hi',
        'F', 'p (F test)', 'CIs overlap']
pd.DataFrame(rows, columns=cols).round(4)
```

Out[5]:

	variable	sampling	var 2015	lo	hi	var 2020	lo	hi	F	p (F test)	CIs overlap
0	temperature	full	6.8096	6.7443	6.8759	6.5122	6.4626	6.5625	1.0457	0.0000	False
1	temperature	daily	7.1447	6.1866	8.3453	6.8639	5.9676	7.9797	1.0409	0.7053	True
2	pressure	full	0.2427	0.2404	0.2451	0.2533	0.2514	0.2553	0.9580	0.0000	False
3	pressure	daily	0.2337	0.2023	0.2729	0.2480	0.2156	0.2883	0.9422	0.5767	True

3. Discussion

Full resolution. The variances appear inconsistent: the confidence intervals do not overlap and p ≈ 0. However, the variance ratios are 1.05 for temperature and 0.96 for pressure. The two years differ by only 4–5%. The intervals are only ±1% wide because the tests assume about 10^5 independent samples. As with the means, this greatly overstates the degrees of freedom.

Daily subsampling. With one sample per day, the intervals widen to about ±15% and overlap. The F-test is not significant, with p = 0.71 for temperature and 0.58 for pressure. So the variances are consistent. Daily temperature samples are still correlated, so the true intervals are even wider. This makes the conclusion stronger.

Interpretation. 2015 was warmer on average, but its spread was not larger. The yearly variance mixes the seasonal cycle with short-term variability. These are distributed differently in the two years, but their totals are similar. Pressure variance is about 0.25 dbar², or σ ≈ 0.5 m. It comes mainly from tides, which are stable from year to year.

4. Probability density functions

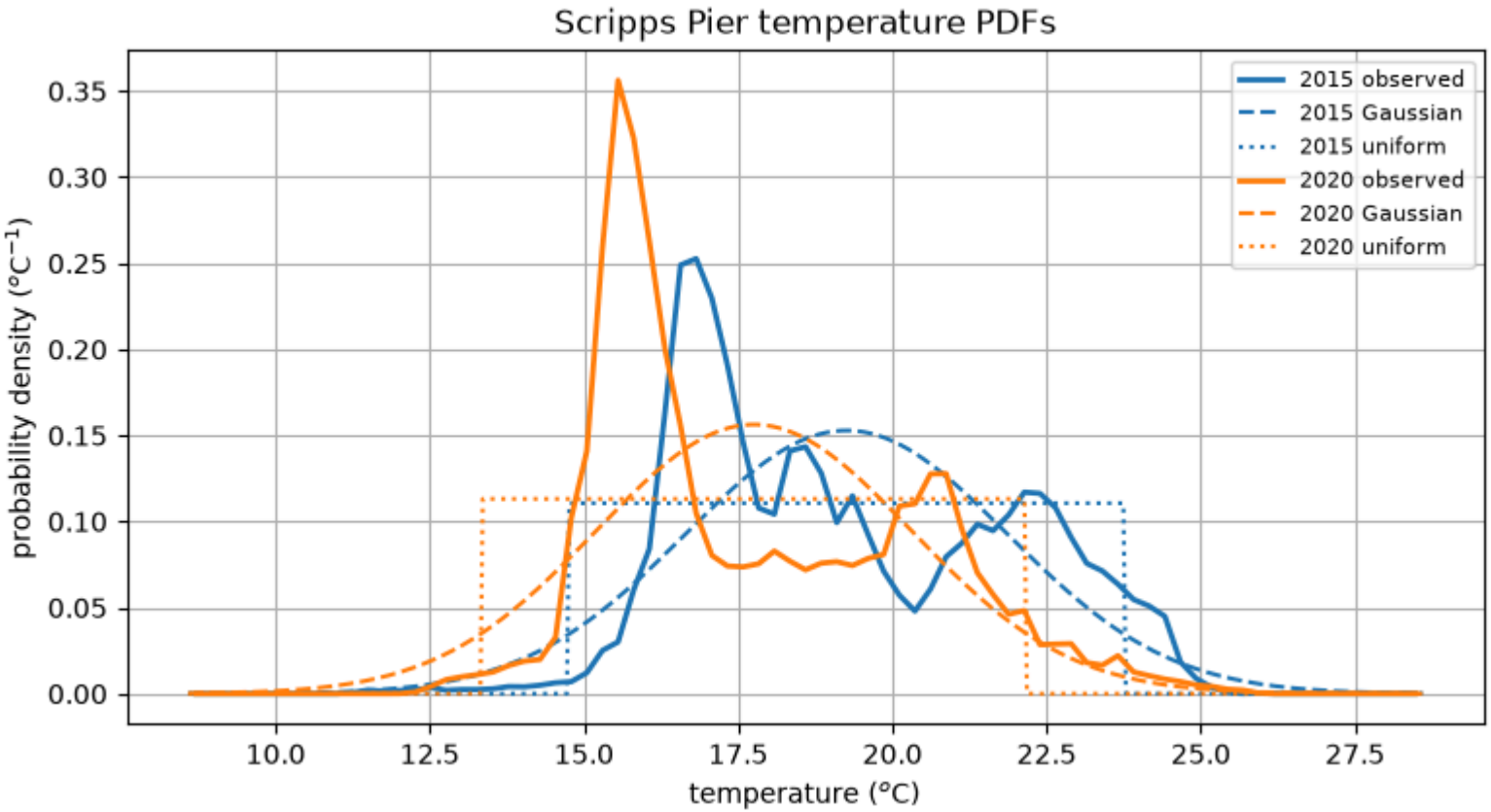
Gaussian and uniform distributions with the observed mean and variance. A uniform distribution with variance σ^2 has width $\sqrt{12}\,\sigma$, i.e. it spans $\mu \pm \sqrt{3}\,\sigma$.

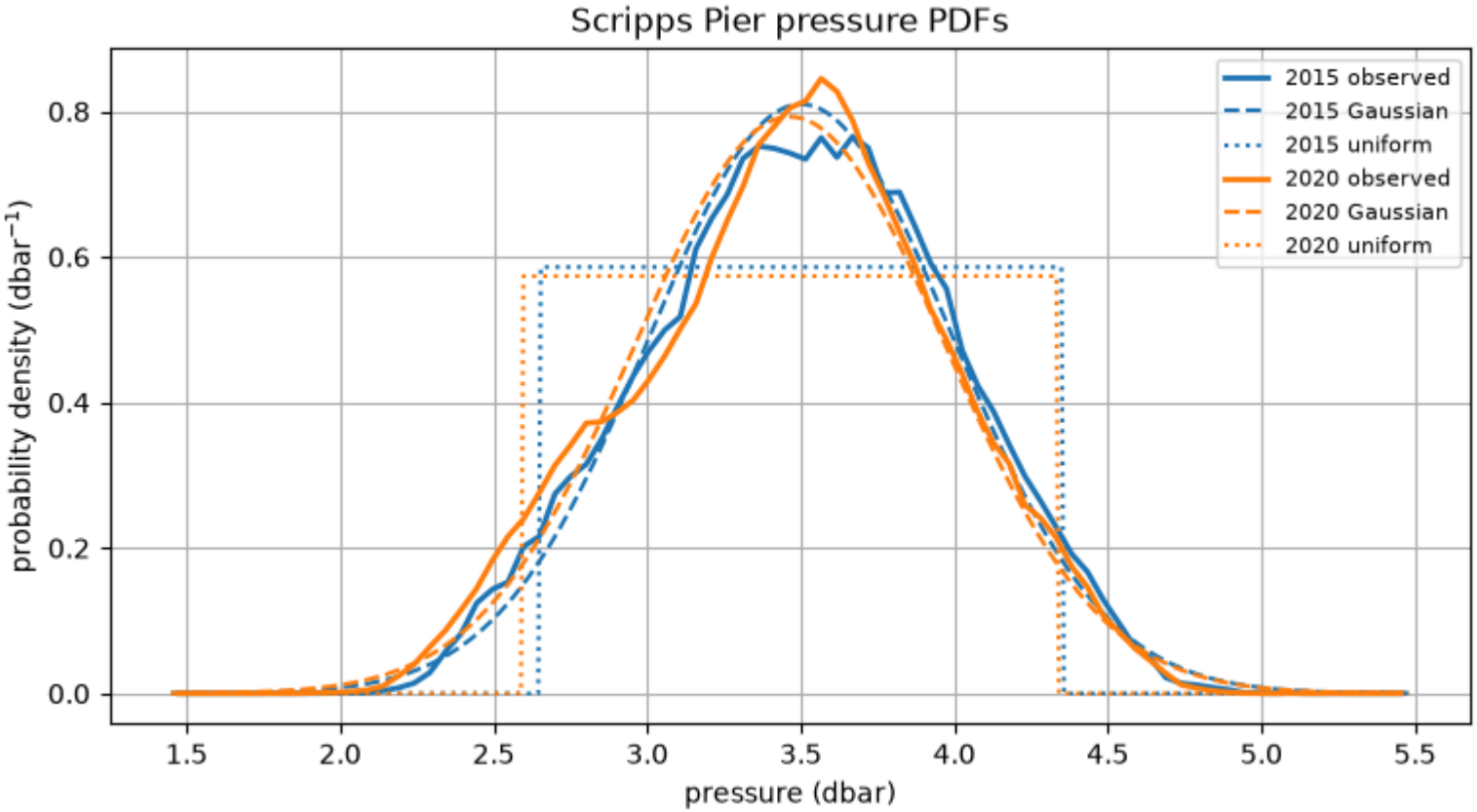
In [6]:
```
def pdf_hist(x, edges):
    counts, _ = np.histogram(x, bins=edges)
    return counts / counts.sum() / np.diff(edges)

colors = {Y1: 'C0', Y2: 'C1'}
for v, (label, unit) in VARS.items():
    allx = np.concatenate([get_series(y, v, 'full') for y in (Y1, Y2)])
    lo, hi = np.percentile(allx, [0.01, 99.99])
    pad = 0.2 * (hi - lo)
    edges = np.linspace(lo - pad, hi + pad, 80)
    centres = 0.5 * (edges[1:] + edges[:-1])
    xx = np.linspace(edges[0], edges[-1], 500)

    fig, ax = plt.subplots()
    for y in (Y1, Y2):
        x = get_series(y, v, 'full')
        mu, sd = x.mean(), x.std(ddof=1)
        ax.plot(centres, pdf_hist(x, edges), color=colors[y], linewidth=2, label=f'{y} observed')
        ax.plot(xx, stats.norm.pdf(xx, mu, sd), '--', color=colors[y], label=f'{y} Gaussian')
        a = np.sqrt(3) * sd
        ax.plot(xx, np.where(np.abs(xx - mu) <= a, 1 / (2 * a), 0), ':', color=colors[y],
                label=f'{y} uniform')
    ax.set_xlabel(f'{label} ({unit})')
```

```
    ax.set_ylabel(f'probability density ({unit}$^{{-1}}$)')
    ax.set_title(f'Scripps Pier {label} PDFs')
    ax.legend(fontsize=8)
    ax.grid(True)
    plt.show()
```





4. Discussion

Temperature. Neither a Gaussian nor a uniform distribution fits. Both PDFs are multimodal and skewed. In 2020, there is a sharp peak near 15.5 °C and a broad plateau from 17 to 22 °C. In 2015, the main peak is near 16.8 °C, with smaller peaks near 18.5 and 22.3 °C. The sharp peaks come from the calm winter months. The 2015 peak is about 1.3 °C warmer, and its 22–23 °C peak reflects the warm fall. The cold side drops off steeply, with only a thin tail to 12 °C. The warm side extends to 25–26 °C. The Gaussian gives too much probability to cold values and misses the peak. The uniform distribution has hard edges and no peak. The yearly record mixes several regimes, so one mean and one variance cannot describe it.

Pressure. Pressure is close to Gaussian, and the two years are nearly identical. This is interesting because a single sine wave has a U-shaped PDF. The tide is a sum of many constituents with different frequencies. By the central limit theorem, their sum tends

toward a Gaussian. Both years have slightly extra probability at low pressure, around 2.3–3.0 dbar. This slight negative skew may come from the mixed tide at San Diego, where one of the two daily low tides is much lower. The uniform distribution fits poorly.

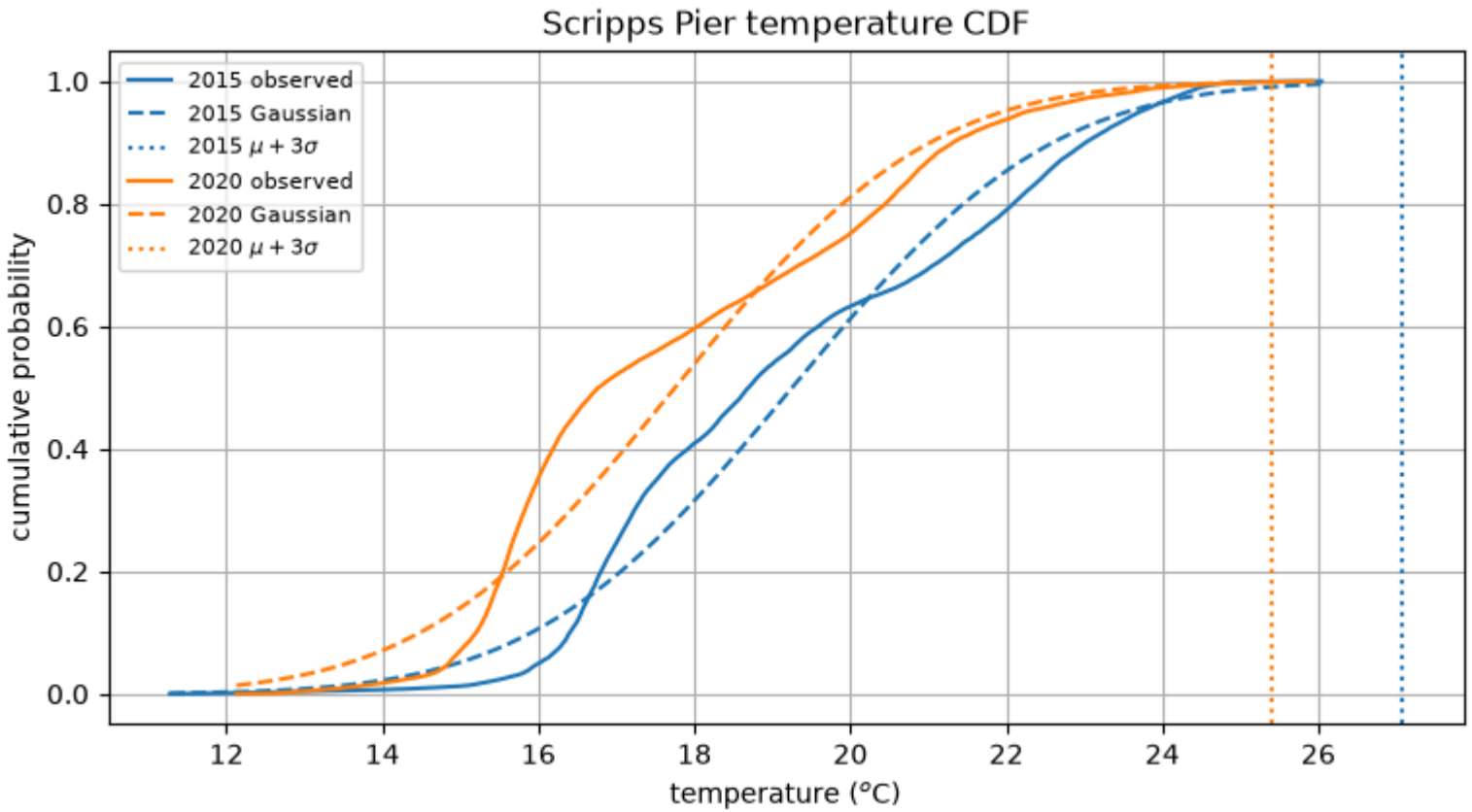
5. Extreme values

Probability of a temperature more than 3σ above the mean, from the observed PDF (fraction of samples) and from a Gaussian with the observed variance ($1 - \Phi(3) \approx 0.00135$).

In [7]:
```
fig, ax = plt.subplots()
for y in (Y1, Y2):
    x = get_series(y, 'temperature', 'full')
    mu, sd = x.mean(), x.std(ddof=1)
    thr = mu + 3 * sd
    p_obs = np.mean(x > thr)
    p_gauss = stats.norm.sf(3)
    print(f'{y}: threshold = {thr:.2f} degC, P_observed = {p_obs:.2e}, P_Gaussian = {p_gauss:.2e}')

    # Empirical CDF vs Gaussian CDF
    xs = np.sort(x)
    ax.plot(xs, np.arange(1, len(xs) + 1) / len(xs), color=colors[y], label=f'{y} observed')
    ax.plot(xs, stats.norm.cdf(xs, mu, sd), '--', color=colors[y], label=f'{y} Gaussian')
    ax.axvline(thr, color=colors[y], linestyle=':', label=f'{y} $\\mu+3\\sigma$')
ax.set_xlabel('temperature ($^o$C)')
ax.set_ylabel('cumulative probability')
ax.set_title('Scripps Pier temperature CDF')
ax.legend(fontsize=8)
ax.grid(True)
plt.show()
```

```
2015: threshold = 27.08 degC, P_observed = 0.00e+00, P_Gaussian = 1.35e-03
2020: threshold = 25.41 degC, P_observed = 1.02e-03, P_Gaussian = 1.35e-03
```



5. Discussion

For a Gaussian, the chance of exceeding μ + 3σ is 1.35×10^{-3}, or about 12 hours per year.

- **2015:** The threshold is 27.08 °C, which is above the maximum observed value of about 26 °C. So the observed probability is zero. The broad, multimodal distribution inflates σ and pushes the threshold beyond the real range.
- **2020:** The threshold is 25.41 °C. The observed probability is 1.02×10^{-3}, about 75% of the Gaussian value. But these samples come almost entirely from one warm event in late August. They represent only one or two independent events, so the estimate is very uncertain.

In both years, the warm tail is lighter than Gaussian. Pier temperature rarely goes far above the summer peak. The cold side shows the opposite: between 12 and 16 °C the Gaussian CDF is above the observed CDF, so the Gaussian overestimates cold values.

2015 was a heatwave year, yet it has no 3σ warm extremes. This is because the threshold is relative to its own warm mean and large σ. Using the 2020 threshold, 2015 would exceed it in early September. So "extreme" depends on the reference.

Implications. A Gaussian fit to a non-stationary, multimodal record is a poor model for extremes. Extremes also cluster in time, so counting samples overstates the information. Better estimates would use seasonal anomalies, longer records and extreme-value methods.

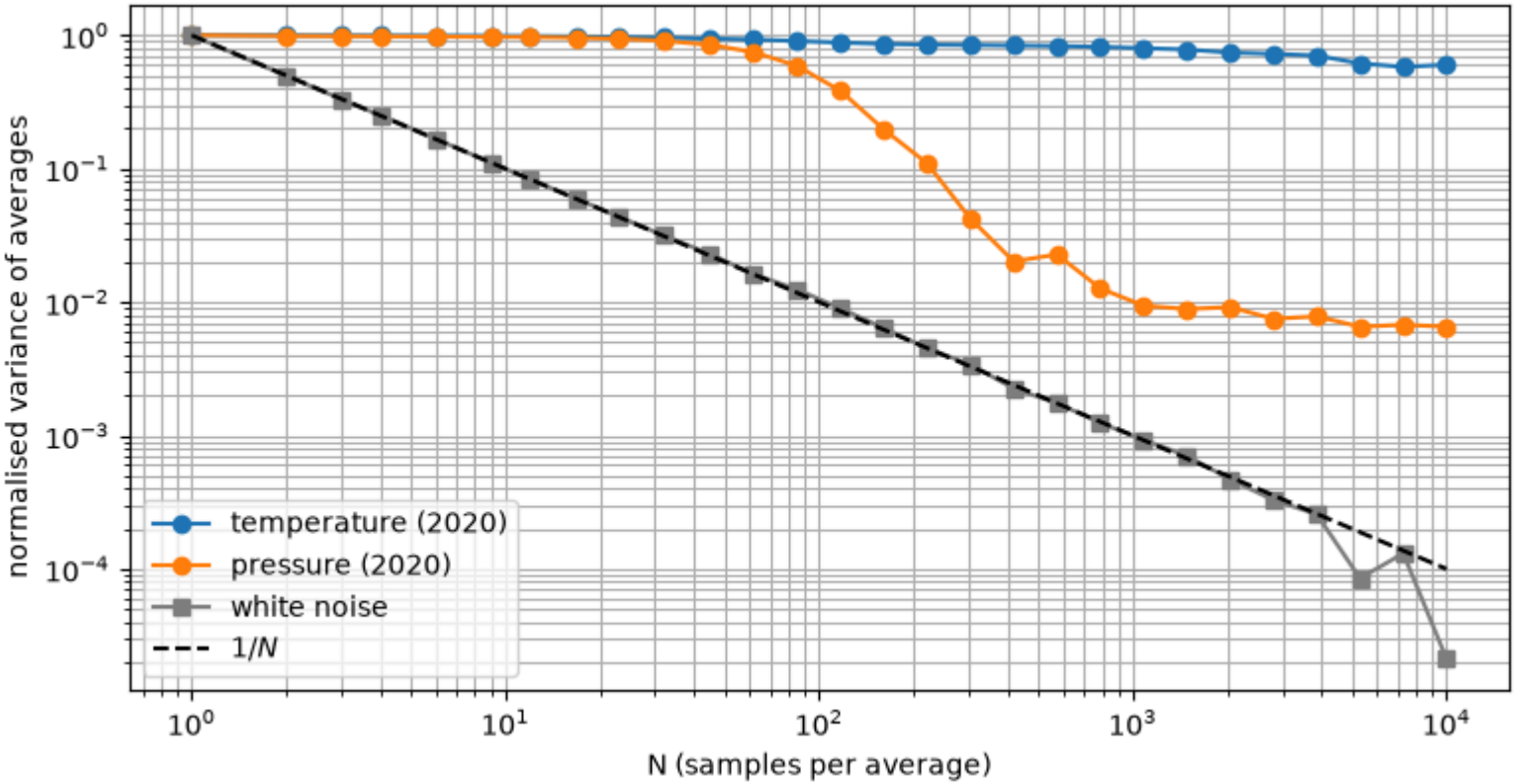
6. Bonus: variance reduction by averaging

Average non-overlapping blocks of N consecutive samples and compute the variance of the block means. For independent samples it should fall as σ^2/N (slope -1 on a log-log plot).

In [8]:
```
Ns = np.unique(np.logspace(0, 4, 30).astype(int))
fig, ax = plt.subplots()
for v, (label, unit) in VARS.items():
    x = get_series(Y2, v, 'full')
    var_N = []
    for N in Ns:
        nb = len(x) // N
        block_means = x[:nb * N].reshape(nb, N).mean(axis=1)
        var_N.append(block_means.var(ddof=1))
    var_N = np.array(var_N)
    ax.loglog(Ns, var_N / var_N[0], 'o-', label=f'{label} ({Y2})')

# Synthetic white noise for comparison
rng = np.random.default_rng(0)
w = rng.standard_normal(len(x))
var_w = [w[:(len(w)//N)*N].reshape(-1, N).mean(axis=1).var(ddof=1) for N in Ns]
ax.loglog(Ns, np.array(var_w) / var_w[0], 's-', color='0.5', label='white noise')
ax.loglog(Ns, 1.0 / Ns, 'k--', label='$1/N$')
ax.set_xlabel('N (samples per average)')
ax.set_ylabel('normalised variance of averages')
ax.legend()
```

```
ax.grid(True, which='both')
plt.show()
```



6. Discussion

At 4-min sampling, N = 15 is 1 hour, N = 186 is one M2 tidal period, N = 360 is one day and N = 10^4 is 28 days.

White noise follows 1/N exactly, as expected for independent samples. The scatter at large N occurs because only a few blocks remain.

Pressure does not follow 1/N. Below N = 50, the variance barely drops because the tide changes smoothly over a few hours. Between N = 100 and 400, it drops by two orders of magnitude, faster than 1/N. Here the averaging window covers a full tidal

cycle, so the tide cancels out. Above N = 1000, the variance levels off at about 0.7% of the total, or $\sigma \approx 4$ cm. This remainder is slow sea-level change from weather and seasons.

Temperature drops to only 60% of its variance even after 28 days of averaging. Most of its variance comes from the seasonal cycle and multi-day events.

Link to Problems 2 and 3. The normalized variance equals $1/N_{eff}$, where N_{eff} is the effective number of independent samples. For temperature, 28 days of data give $N_{eff} \approx 1.7$. A year therefore has roughly 20 independent samples. With this value, the standard error in Problem 2 becomes about 0.6 °C and $z \approx 1.8$. So the difference between the 2015 and 2020 means is only marginally significant. This estimate is rough because the record is not stationary. Still, it shows that correlation between samples, not the amount of data, controls the uncertainty.